PLATAFORMA DE CURSOS

Módulo de cursos, inventario de operaciones y pruebas de API

1. Objetivo

Desarrollar y documentar el módulo de cursos del backend de la Plataforma de Cursos. La entrega incluye las seis operaciones asignadas, la actualización del inventario de 30 operaciones y una colección de Postman con escenarios para revisar las respuestas de la API.

2. Alcance del módulo

El módulo permite consultar el catálogo de cursos, obtener un curso por su ID, buscar y filtrar cursos, crear cursos, modificar su información y eliminarlos. De acuerdo con la organización del equipo, las tres operaciones de lectura usan SQL nativo mediante pg y las tres operaciones de escritura utilizan Sequelize.

ID	Método y endpoint	Operación	Acceso a datos	Acceso
CUR-01	GET /api/courses	Listar cursos	SQL nativo (pg)	Público
CUR-02	GET /api/courses/:id	Consultar por ID	SQL nativo (pg)	Público
CUR-03	GET /api/courses/search	Buscar y filtrar	SQL nativo (pg)	Público
CUR-04	POST /api/courses	Crear curso	Sequelize	JWT
CUR-05	PUT /api/courses/:id	Actualizar curso	Sequelize	JWT
CUR-06	DELETE /api/courses/:id	Eliminar curso	Sequelize	JWT

3. Estructura de archivos entregada

- **src/models/Course.js:** Define el modelo Course y relaciona sus campos con la tabla courses.
- **src/controllers/course.controller.js:** Contiene las seis operaciones y las validaciones principales.
- **src/routes/course.routes.js:** Declara los endpoints y protege las operaciones de escritura con JWT.
- **postman/Plataforma_Cursos.postman_collection.json:** Colección importable en Postman.
- **Inventario_operaciones_backend.xlsx:** Conserva las 30 operaciones originales y agrega una hoja con el detalle.
- **INTEGRACION-server.js.txt y README.md:** Indican cómo integrar los archivos al backend común.

4. Funcionamiento de las operaciones

Listar cursos devuelve los registros ordenados por ID e incluye el nombre de la categoría. Consultar por ID valida que el identificador sea positivo y responde con 404 cuando el curso no existe. La búsqueda acepta el parámetro q para buscar por título o descripción, y filtros opcionales category_id, min_price y max_price. También valida rangos de precio y evita que el precio mínimo sea mayor que el máximo.

Crear curso requiere un título y una categoría, y acepta descripción y precio. Actualizar curso modifica únicamente los campos enviados. Eliminar curso devuelve un mensaje de confirmación o 404 si no encuentra el registro. Las operaciones de escritura requieren enviar un token JWT en el encabezado Authorization con el formato Bearer.

5. Formato de respuestas

Las respuestas siguen el formato usado por el módulo de categorías: success indica si la operación fue correcta y data contiene el registro o la lista. Cuando ocurre un error de validación se devuelve un mensaje claro; cuando un curso no existe se utiliza el estado HTTP 404. Los errores inesperados se envían al middleware central del servidor.

6. Escenarios de prueba propuestos

Caso	Solicitud	Resultado esperado
T01	GET /api/courses	Debe responder 200 y entregar data como arreglo.
T02	GET /api/courses/1	Con un ID existente debe responder 200; el ID puede ajustarse a los datos cargados.
T03	GET /api/courses/999999	Si no existe debe responder 404.
T04	GET /api/courses/search?q=PostgreSQL	Debe devolver coincidencias en título o descripción.
T05	GET /api/courses/search?category_id=2&min_price=10&max_price=60	Debe aplicar los filtros enviados.
T06	GET /api/courses/search?min_price=80&max_price=10	Debe responder 400

		porque el rango es inválido.
T07	POST /api/courses sin token	Debe responder 403 por falta de token.
T08	POST /api/courses con token y datos válidos	Debe responder 201 y entregar el curso creado.
T09	POST /api/courses con precio negativo	Debe responder 400.
T10	PUT /api/courses/:id con token	Debe actualizar los campos enviados; probar también un ID inexistente (404).
T11	DELETE /api/courses/:id con token	Debe responder éxito si existe y 404 si no existe.
T12	PUT/DELETE sin token	Debe responder 403.

7. Pasos para integrar y probar

- Copiar los archivos de src/models, src/controllers y src/routes al proyecto del equipo, sin sobrescribir los archivos de las demás integrantes.
- Añadir en src/server.js la importación de course.routes y registrar app.use("/api/courses", courseRoutes). Las líneas están en INTEGRACION-server.js.txt.
- Verificar que config/database.js exporte sequelize y pool, y que .env contenga DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD y JWT_SECRET.
- Confirmar que database/schema.sql y database/seed.sql se hayan ejecutado en PostgreSQL.
- Iniciar el backend e importar la colección JSON en Postman. Configurar baseUrl como http://localhost:3000 y pegar un token válido en la variable token para las solicitudes protegidas.
- Ejecutar cada caso y registrar el código HTTP y la respuesta real en la evidencia de entrega.

8. Consideraciones y pendientes

Los archivos se prepararon tomando como base el esquema SQL y el código compartido. En este entorno no se ejecutaron pruebas conectadas a la base de datos real; por eso los resultados anteriores son esperados, no resultados medidos. Antes de dar por terminada la integración, hay que confirmar que el JWT incluya el identificador del usuario como id, ya que el módulo lo usa para guardar created_by. La configuración compartida muestra una diferencia entre DATABASE_URL en el ejemplo y DB_* en config/database.js; para este módulo se documentan las variables DB_* que consume ese archivo. Finalmente, el código de categorías protege escrituras con verifyToken, por lo que el módulo sigue ese patrón; el equipo debe decidir si en una versión posterior exige además permisos de administrador o instructor.

9. Conclusión

Esta parte deja preparado el módulo de cursos con seis endpoints, una distribución clara entre SQL nativo y Sequelize, el inventario consolidado y escenarios para comprobar el funcionamiento. La entrega queda lista para integrarse al backend común, aunque la comprobación final depende de ejecutar las pruebas en el entorno de PostgreSQL y con un token válido.